# Task 3: Apache Web Server – Installation and Virtual Hosting Configuration

## Objective

Install Apache web server on your local machine
Create an index.html page in the default document root
Access the website using "mysamplewebsite.itfs"
Create a custom Virtual Hosting configuration file
Keep a backup of system configuration files before editing
Study in detail about Apache default files, document root, and modules

## Requirements

- AWS EC2 instance running RHEL
- Apache (httpd) web server
- A simple index.html webpage
- A custom Virtual Host configuration file
- AWS Security Group configuration
- Windows client machine for browser testing

## Procedure

## Step 1 – Launch the EC2 Instance

An AWS EC2 instance was launched to serve as the RHEL machine for hosting the Apache web server.

## Step 2 – Install Apache

The Apache web server package was installed on the RHEL EC2 instance using the dnf package manager.

```
sudo dnf install httpd -y
```

After installation, the Apache service was enabled so that it starts automatically on system boot.

```
sudo systemctl enable --now httpd
```

The status of the service was verified using:

```
sudo systemctl status httpd
```

The Apache service was confirmed to be active and running.

## Step 3 – Create index.html in the Default Document Root

Apache's default document root on RHEL was identified as /var/www/html.

The index.html file inside this default document root was created and edited with simple custom webpage content.

```
sudo vi /var/www/html/index.html
```

## Step 4 – Back Up Configuration Files Before Editing

Before creating any Virtual Host configuration, a backup of the existing Apache configuration files was taken, to allow safe rollback in case of error.

```
sudo cp /etc/httpd/conf/httpd.conf /etc/httpd/conf/httpd.conf.bak

sudo cp -r /etc/httpd/conf.d /etc/httpd/conf.d.bak
```

The backups were verified to exist before any configuration file was modified.

## Step 5 – Create a Custom Virtual Hosting Configuration File

A new Virtual Host configuration file was created to associate the custom hostname with the website's document root.

```
sudo vi /etc/httpd/conf.d/mysamplewebsite.conf
```

The following configuration was added:

```
<VirtualHost *:80>
   ServerName mysamplewebsite.itfs
   DocumentRoot /var/www/html

   <Directory /var/www/html>
      AllowOverride All
      Require all granted
   </Directory>
</VirtualHost>
```

## Step 6 – Test and Apply the Configuration

```
sudo apachectl configtest

sudo systemctl reload httpd
```

The configuration test returned "Syntax OK," confirming the Virtual Host file contained no errors, and Apache was reloaded to apply the change.

## Step 7 – Test Using the Public IP Address

The webpage was first tested directly using curl on the EC2 instance itself:

```
curl http://localhost
```

The custom HTML content was returned successfully, confirming that Apache was serving the updated index.html file correctly.

The public IP address of the EC2 instance was then tested to confirm the website was also reachable externally:

```
curl http://<EC2-Public-IP>
```

The same custom webpage content was returned successfully, confirming Apache was correctly serving the website over the network and that the AWS Security Group was permitting inbound HTTP traffic on port 80.

## Step 8 – Configure the Custom Hostname (Local DNS Override)

To access the website using the required hostname mysamplewebsite.itfs, the concept of a local DNS override using the hosts file was used, since mysamplewebsite.itfs is not a registered public domain.

On the Windows client machine, the hosts file was edited with administrator privileges:

```
C:\Windows\System32\drivers\etc\hosts
```

The following entry was added, mapping the EC2 instance's public IP address to the custom hostname:

```
<EC2-Public-IP>   mysamplewebsite.itfs
```

This allowed the Windows machine to resolve mysamplewebsite.itfs directly to the Apache server's IP address, bypassing real DNS entirely.

## Step 9 – Verify the Custom Domain

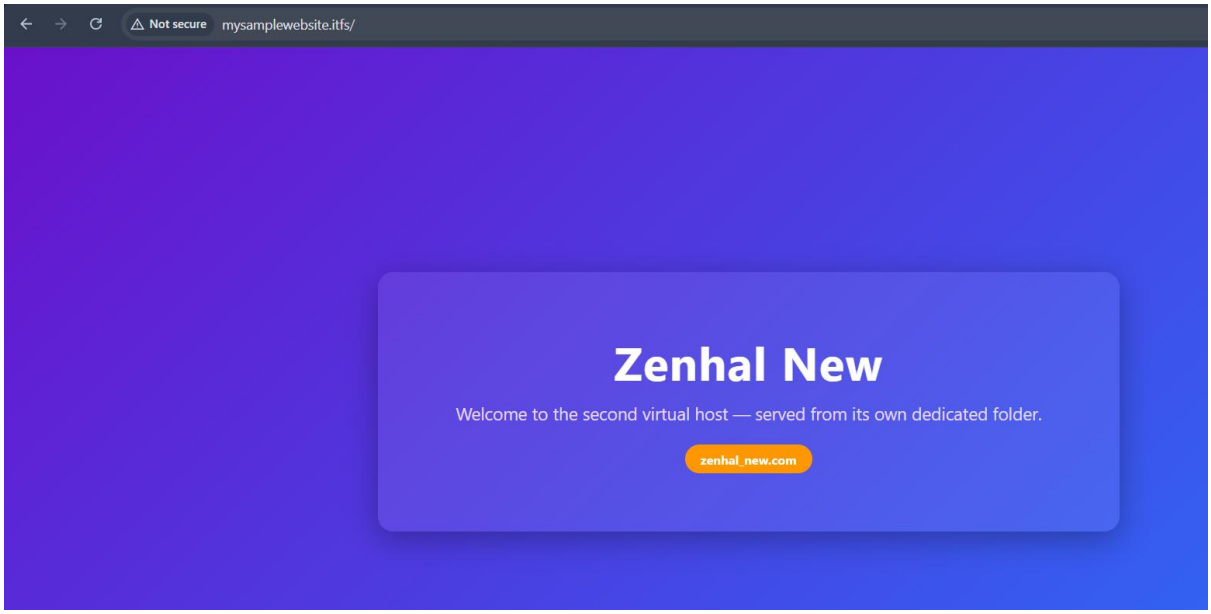
The custom hostname was verified using curl with a manually set Host header on the server:

```
curl http://localhost -H "Host: mysamplewebsite.itfs"
```

It was then tested by entering the following address into the Windows web browser:

```
http://mysamplewebsite.itfs
```

The custom webpage content was displayed successfully, confirming that the website was accessible using the custom hostname and the custom Virtual Host configuration.




## Verification

- The Apache package was successfully installed on the RHEL EC2 instance.
- The Apache service was started and enabled to run automatically on boot.
- The default document root used by Apache was identified as /var/www/html.
- The index.html file was created with custom webpage content.
- Configuration files were backed up before any modifications were made.
- A custom Virtual Host configuration file was created to associate the hostname mysamplewebsite.itfs with the website.
- The AWS Security Group was verified to allow inbound HTTP traffic on port 80.
- The website was successfully reached using the EC2 instance's public IP address.
- The custom hostname mysamplewebsite.itfs was set up using a local hosts file entry on the Windows machine.
- The hosted webpage was successfully displayed in the Windows browser using mysamplewebsite.itfs.

## Conclusion

Through this task, the Apache web server was successfully installed and configured on an AWS EC2 instance running RHEL. A simple website was hosted in Apache's default document root and made accessible using a custom hostname, mysamplewebsite.itfs, through a custom Virtual Host configuration file. Configuration files were backed up prior to editing as a safety precaution. The AWS Security Group was checked to ensure external HTTP access was permitted, and a local hosts file entry was used to map the custom hostname to the server. This task provided practical, hands-on experience with Apache installation, Virtual Host configuration, safe configuration management practices, and Apache's internal file and module structure.